arXiv:2510.07080v1 [cs.CR] 8 Oct 20

stage stochastic control and in applications such as taxi dispatch and the Last Revealer Attack (LRA) on Ethereum's RANDAO. We focus on the LRA, which undermines fairness in certain Proof-of-Stake (PoS) blockchains such as Ethereum ($400B market capitalization). Standard modeling of this problem as an MDP would lead to a state space with a size of the order of $10^{10^{10}}$ and stochastic inputs as large as 5 GB, making it intractable. We introduce the notion of pseudo-MDP (pMDP), a framework that naturally models such problems and propose two distinct reductions to standard MDPs. The duality of the reductions provides a novel perspective, enabling significant improvements in value iteration algorithms, including a complexity compression close to logarithmic for the dynamic programming implementation and a more efficient Monte Carlo variant. In the case of the LRA which size is parameterized by κ (in Ethereum's case $\kappa = 32$), we reduce the computational complexity per iteration from $O\left(2^\kappa \kappa^{2^{\kappa+2}}\right)$ to $O\left(\kappa^4\right)$. This perspective also simplifies policy extraction, making the approach well-suited for resource-constrained agents, once again for the LRA, we lower the worst-case complexity from: $O\left(2^\kappa \kappa^{2^{\kappa+1}}\right)$ to $O\left(2^\kappa\right)$. The application case of the LRA demonstrates the capability of the pMDP framework to solve large scale problems effectively. Our approach also provides the usual benefits from dynamic programming solutions: exponentially fast convergence toward the optimal solution is **guaranteed**. The efficiency we achieve comes at the cost of the framework's applicability, which is why we also provide an extension of some results to a broader class of MDPs than that of reduced pMDPs.

KEYWORDS

Markov Decision Processes (MDPs), Proof-of-Stake (PoS) Blockchain security, RANDAO, Random Seed Consensus, Last Revealer Attack (LRA), Dynamic Programming, Value Iteration, Policy Extraction, Resource-Constrained Agents

ACM Reference Format:

Maxime Reynouard. 2026. Pseudo-MDPs: A Novel Framework for Efficiently Optimizing Last Revealer Seed Manipulations in Blockchains. In *Proc. of the 25th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2026), Paphos, Cyprus, May 25 – 29, 2026*, IFAAMAS, 15 pages.

Proc. of the 25th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2026), C. Amato, L. Dennis, V. Mascardi, J. Thangarajah (eds.), May 25 – 29, 2026, Paphos, Cyprus. © 2026 International Foundation for Autonomous Agents and Multiagent Systems (www.ifaamas.org). This work is licenced under the Creative Commons Attribution 4.0 International (CC-BY 4.0) licence.

energy [2, 10], Proof of Stake (PoS) blockchains select consensus participants based on their stake. To guarantee fairness and security, PoS protocols rely on reliable randomness sources. Several prominent PoS blockchains [11, 13, 17] adopted the RANDAO (Random Autonomous Organization) protocol [19], which generates randomness through a decentralized, verifiable two-step *commit-reveal* process [12]: participants first commit secret values via hashes, then reveal them to collectively produce the random outcome (typically using XOR). We specifically examine Ethereum's RANDAO due to Ethereum's prominence and market cap in the hundreds of billions of USD.

Time-constrained commit-reveal schemes are vulnerable to the Last Revealer Attack (LRA) [24], where the final participant influences the outcome by choosing whether or not to reveal their secret, enabling selection between the intended result or a fallback value. This undermines the fairness and unpredictability of randomness generation. Although widely recognized in cryptographic research, pinpointing the origin of this vulnerability is challenging. Several mitigation strategies exist, such as incentive design, Threshold Schemes [21], or Verifiable Delay Functions [1, 7], but exploitation remains a credible threat [24]. Until recently, quantifying the advantage from LRA was difficult [3]. The existing solution, while innovative, applies only specifically to RANDAO's LRA and is not easily generalized. Our approach, in addition to being general, provides a slight algorithmic improvement, reducing complexity from $O\left(\kappa^5\right)$ to $O\left(\kappa^4\right)$.

Markov Decision Processes (MDPs) [5, 18] are natural tools to model and optimize this attack, but the combinatorial complexity makes their use challenging — Ethereum's case would require a prohibitive number of states ($\in O\left(\kappa^{2^{\kappa+1}}\right)$). Factored MDPs [8] could improve representation, though they still involve products of 2^κ sets and do not simplify solving the MDP. Standard dynamic programming methods solve small MDPs efficiently but struggle at scale. Heuristic or approximate methods like Monte Carlo Tree Search or neural-network-based function approximation can manage larger state spaces but cannot effectively exploit structure due to the Cryptographically Secure PseudoRandom Number Generators (CSPRNG) [14], which guarantee high entropy. Additionally, such approaches only yield approximate solutions without strong theoretical guarantees.

The difficulty of modeling RANDAO's LRA using MDPs stems from an inversion in the natural flow of decisions. Traditionally,

state s; by opposition to the *ex-ante state* σ before the stochastic process. Such a structure is vaguely similar to that of two-stage stochastic control problems [23] where agents take decisions in two steps. When such problems are repeated, we can find other applications to our framework. For instance, in the case of ride selection for taxis: from a certain area σ (ex-ante state), drivers are stochastically offered several taxi rides from their various apps and dispatch $\mathcal{A}$, they select one which yields a fare R and takes them to a new area σ'. It seems intuitive that there might be situations where it is worthwhile for a driver to accept a lower fare than other offers (adjusted for the ride time), if it takes them to a better area (in the sense that it will probabilistically lead to better ride proposal). This is the usual trade-off agents face in MDPs: instant reward or higher delayed reward. The distinction between *ex-ante* states (before the stochastic process) and *ex-post* states (after the stochastic process, matching standard MDP) also naturally arises in certain games, such as *Seasons* (players choose which dice to roll, that is they choose an ex-ante state; the dice roll result form the ex-post state) and *Dice Forge* (players modify dice faces thus changing future ex-ante states).

Main contributions: The main technical contributions of the paper are:

- The introduction of *pMDPs*, as a complementary discrete-time stochastic control process to that of MDPs [18]. They naturally model the reversed decision flow of the LRA. In doing so they fully solve the problem representation issue.
- Show that pMDPs can be reduced to MDPs in two different ways. The reductions allows us to get all the known results of MDPs such as known solutions and their guarantees [22]. But they do not solve the scalability issue: For instance the complexity of value iteration is in $O\left(2^{\kappa}\,\kappa^{2^{\kappa+2}}\right)$ per iteration.
- Lowering the complexity of value iteration in two steps, ending up in $O\left(\kappa^4\right)$ by combining the strengths of the two problem reductions.
- Extending those results to a larger class of MDPs where *ex-ante states* act as intermediate steps in the Markov transitions.

combined to produce a pseudo-random seed which will be used to generate the slot allocation at the next epoch.

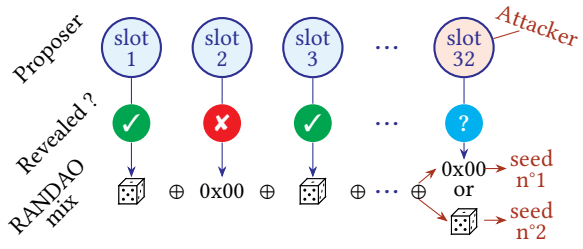

Figure 1: Illustration of Ethereum RANDAO LRA

To clarify the LRA, we examine the revelation phase (see Figure 1). During an epoch, block proposers must include their revealed secret numbers, or else their blocks are ignored. Thus, each slot results either in a block with the revealed value (green check-mark), or no block (red "x"). The resulting random seed, computed by XOR-ing revealed values (missing reveals default to 0), determines block proposal rights for the *next* epoch (of the same parity). The last revealer can decide whether to propose (revealing their secret, yielding seed n°2) or withhold (yielding seed n°1). The attacker then selects the seed yielding the most favorable block allocations. As detailed in [24], several incentives motivate this choice; here, we focus solely on rewards from accepted block proposals, normalized to 1 per block. One must also take into consideration that withholding a block results in an opportunity cost of 1 since no reward will be gained. Consider this simple example:

Example 2.1. The agent can either 1) propose a block, yielding random seed n°1 with 2 block proposal rights in the next epoch, or 2) withhold the block (possibly disguised as a network issue), yielding random seed n°2 with 5 proposal rights next epoch. Although strategy 2) costs 1 block proposal immediately, it ultimately yields 3 additional rights in the following epoch, making it the rational choice.

Now suppose the attacker controls the final 2 blocks of an epoch. He can choose between proposing both, hiding then proposing, proposing then hiding, or hiding both—giving him a choice among

[1]Agents stake part of their Ethereum currency to participate in consensus. This staked currency is temporarily frozen as collateral to penalize misbehavior.

4 random seeds. More generally, controlling the last k blocks provides a choice among 2^k seeds. While prior analyses typically stop at this first-order attack [24], controlling the final blocks in epoch e enhances opportunities in epoch $e+1$, making these blocks inherently more valuable. Thus, the attacker faces a decision problem: maximize immediate rewards or strategically position himself to increase future block control and rewards. Consider this example:

Example 2.2. The agent controls the last block of the epoch. If he 1) proposes, he receives 2 proposal rights next epoch (including final slots); if he 2) hides, he receives 4 rights but none at epoch's end. These choices yield net rewards of 2 and 3, respectively. However, strategy 1) offers control over the next epoch's final blocks, possibly justifying a short-term loss if expected future rewards exceed 1.

2.2 Example: A simple card game

We introduce a fictional card game designed to illustrate the above mechanism, it makes the concepts discussed throughout this paper more intuitive. Table 1 illustrates the game. We assume an infinite number of standard decks shuffled together, ensuring each card draw is uniform and IID (identical cards can appear consecutively). Each round proceeds as follows:

- The dealer draws d cards (the first round, $d = 2$)
- The player chooses card i
- He earns the value of the card (figures are valued at 10)
- He pays 7 if he picked the first card, 8 if he picks the second one, and so on
- The dealer clears the cards, and depending on the color of the picked card (club, spade, diamond, heart) we respectively set $d = 1, 2, 4, 8$ for the next round.

Representing this problem as an MDP requires capturing the full state information before actions are chosen. With up to 8 cards on the table, there are more than 40^8 possible states (40 instead of 52 since all face cards count as 10). Given $d \in 1, 2, 4, 8$, this results in $40 + 40^2 + 40^4 + 40^8$ total states, causing a combinatorial explosion typical of MDP formulations.

3 MODELING

We now present our notations for MDPs, and then introduce *pseudo MDPs* (pMDPs) and explain the two problem reductions (ex-post and ex-ante). We use the notations $\mathbb{N}$, $\mathbb{N}^*$, $\mathbb{N}_n$ and $\mathbb{N}^*_n$ for the sets of natural integers with/without 0 (with the *), and bounded by n (with $_n$).

can then define $\mathcal{A}(s) = \mathbb{N}^*_{d(s)}$. For each $(s, a, s') \in \mathcal{S} \times \mathcal{A}(s) \times \mathcal{S}$ we have:

$$P(s'|s,a) = \mathbb{1}\left[d(s') = d(s_a^\sigma)\right] \prod_{i \in \mathbb{N}^*_{d(s')}} P(s'_i)$$

with $\mathbb{1}$ [.] being the indicator function and for each $s'_i \in \mathcal{S}_o$ the probability $P(s'_i)$ is $1/52$ if $s'^r_i < 10$ and $4/52$ otherwise (for each color, 4 cards are valued 10). This comes from the fact that when picking action a from state s we know that we will draw $d(s_a^\sigma)$ cards, then those $d(s')$ cards are drawn IID. Finally, for $(s, a) \in \mathcal{S} \times \mathcal{A}(s)$, we would have a net reward $R(s, a) = s_a^r - c(a)$ (here $c(a)$ represents the cost of choosing card a, we have $c(a) = 6 + a$); additionally, we will note this net reward $s_a^R = R(s, a)$. This would be the natural way to represent the card game as an MDP, it is rather complex and the size of S could prevent its solve though standard dynamic programming solutions. Note that in the rest of the paper we keep using r as a reward from a card value and R as a net reward taking into account the cost c.

3.2 Pseudo Markov Decision Process

Before the drawing of the cards for a new round, when we already know d, we could consider ourselves as being in an *ex-ante state*. pMDPs provide a framework describing our problems more intuitively from this ex-ante state.

Definition 3.2. A pMDP is a tuple $(\Sigma, \mathcal{R}, P, d, c)$: Starting from the ex-ante state $\sigma \in \Sigma$, we realize an IID draw of $d : \Sigma \to \mathbb{N}^*$ pairs $(\sigma'_i, r_i)_{i \in \mathbb{N}^*_{d(\sigma)}} \in \mathcal{S}_\sigma$ with probabilities $P(\sigma', r|\sigma)$ where $\mathcal{S}_\sigma = \mathcal{S}_o^{d(\sigma)}$ with $\mathcal{S}_o = \Sigma \times \mathcal{R}$. The agent then picks an outcome $i \in \mathbb{N}^*_{d(\sigma)}$, pays the cost $c(i)$ and earns r_i; The next ex-ante state is σ'_i.

We overload P to represent different probability functions and distributions, context and variables prevent ambiguity.

Card game's case: σ is the color of the previously picked card ($\Sigma = \{$club, spade, diamond, heart$\}$); For rewards $\mathcal{R} = \mathbb{N}^*_{10}$; As before, the uniform draw of a card (over the 52 cards) yields $P(\sigma', r|\sigma)$, that is $4/52$ if $r = 10$ and $1/52$ otherwise ; the picked card's color leads to a state $\sigma' \in \Sigma$. The function d respectively associates $1, 2, 4, 8$ to club, spade, diamond, heart and the cost is $c(i) = 6 + i$. Already we can see how much more naturally pMDPs model such a problem.

randomness: the *ex-post MDP*. The second keeps the smaller *ex-ante* state set as the MDP's state space. **Both** reductions are essential to the understanding of the remainder of the paper.

Ex-post MDP:. For a pMDP $(\Sigma, \mathcal{R}, P, d, c)$, we set the state set as the possible ex-post joint draws: $\mathcal{S} = (\Sigma \times \mathcal{R})^{d(\Sigma)}$, for $s \in \mathcal{S}$, the action set is $\mathcal{A}(s) = \mathbb{N}^*_{d(s)}$. For $(s, a, s') \in \mathcal{S} \times \mathcal{A}(s) \times \mathcal{S}$, since for pMDPs the random process generating s' only depends on $\sigma' = s'^{\sigma}_a$ (we use the same notations as in the card game MDP in section 3.1) , the joint probability of drawing the $(s'^{\sigma}_i, s'^{r}_i)$ pairs from s^{σ}_a is $P(s'|s, a) = P(s'|s^{\sigma}_a) = \prod_{i \in \mathbb{N}^*_{d(s')}} P(s'^{\sigma}_i, s'^{r}_i | s^{\sigma}_a)$ if $d(s') = d(s^{\sigma}_a)$, and 0 otherwise. Finally we set $R(s, a) = s^R_a = s^r_a - c(a)$. The MDP $(\mathcal{S}, \mathcal{A}, P, R)$ is the ex-post reduction.

Ex-ante MDP:. For a pMDP $(\Sigma, \mathcal{R}, P, d, c)$, we keep the state space Σ. The action set is the partial strategies for ex-post states accessible from $\sigma \in \Sigma$, that is: $\mathrm{A}(\sigma) = \{\alpha : s \in \mathcal{S}_\sigma \mapsto s_i \in \mathcal{S}_o,\ i \in \mathbb{N}^*_{d(\sigma)}\}$. For $\alpha \in \mathrm{A}(\sigma)$, we note α^{σ} and α^{r} the projections of α on Σ and $\mathcal{R}$, and α^R the cost adjusted version of α^r. For $(\sigma, \alpha, \sigma') \in \Sigma \times \mathrm{A}(\sigma) \times \Sigma$ the transition probability is $P(\sigma'|\sigma, \alpha) = \sum_{s \in \mathcal{S}_\sigma} P(s|\sigma) \mathbb{1}\left[\alpha^{\sigma}(s) = \sigma'\right]$ with the joint probability $P(s|\sigma) = \mathbb{1}\left[d(s) = d(\sigma)\right] \prod_{i \in \mathbb{N}^*_{d(s)}} P(s^{\sigma}_i, s^{r}_i | \sigma)$ (details in footnote[2]). Finally we set $\mathrm{R}(\sigma, \alpha) = \mathbb{E}(\alpha^R(s)|\sigma)$, for the same joint probabilities $P(s|\sigma)$. This gives us the ex-ante MDP: $(\Sigma, \mathrm{A}, P, \mathrm{R})$.

Set sizes: Table 2 provides a comparison of the different set sizes. We can see that in the ex-post reduction, the complexity is absorbed by the state set $\mathcal{S}$, while in the ex-ante reduction, it is in the action set A. Note that for Ethereum's RANDAO, since $\kappa = 32$, the ex-post state size is of the order of $10^{10^{10}}$, this makes the problem completely intractable for standard dynamic programming algorithms. For comparison, the go game state space is bounded by 3^{19^2}. Additionally, the actions available, in the worst case of the attacker controlling the 32 blocks of the epoch would be described by 2^{32} pairs of (σ', r), that is a roughly 5GB input size.

[2]We decompose over the possible s reached between σ and σ', this gives us: $P(\sigma'|\sigma, \alpha) = \sum_{s \in \mathcal{S}_\sigma} P(\sigma', s|\sigma, \alpha) = \sum_{s \in \mathcal{S}_\sigma} P(s|\sigma, \alpha) P(\sigma'|s, \sigma, \alpha)$. Then we use the fact that the drawing of s only depends on σ and not α to have $P(s|\sigma, \alpha) = P(s|\sigma)$; We also use the fact that from s, the transition to σ' is deterministic and needs $\sigma' = s^{\sigma}_a$. And given s and α we have $s^{\sigma}_a = \alpha^{\sigma}(s)$. Those two points yield: $P(\sigma'|\sigma, \alpha) = \sum_{s \in \mathcal{S}_\sigma} P(s|\sigma) \mathbb{1}\left[\alpha^{\sigma}(s) = \sigma'\right]$.

solve MDP. It is based on iterating the Bellman operator (equation 1, see [6] for details) to find state values. With the state values, *Policy Extraction* provides an optimal policy. We look at how those Algorithms can be adapted to reduced pMDPs, increasing their performance. Furthermore, we provide necessary conditions extending those results to a larger sub-class of MDPs. Finally, we show how the approach can be used in Monte Carlo Value Iteration (MCVI).

4.1 MDPs

Value Iteration: (pseudocode in appendix 10, algorithm 1), computes a *value function* which assigns a value to each state when following a given policy $\pi : s \in \mathcal{S} \mapsto a \in \mathcal{A}(s)$. This value is the expected actualized sum (for a given a *discount factor* γ) of the rewards an agent gets when starting in that state: $V_\pi(s) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_t | s_0 = s, \pi\right]$ where s_t and R_t are the state and reward at time $t \in \mathbb{N}$. An optimal policy π^*, produces the optimal value function $V^*(s)$. The Bellman Operator T^* admits V^* as its fixed point:

$$T^*V(s) = \max_{a \in \mathcal{A}(s)} \left[R(s, a) + \gamma\, \mathbb{E}(V(s')|s, a)\right] \tag{1}$$

This operator being contracting, iterating it over any initial V converges exponentially fast to V^*. However, the complexity (see appendix 10) of an iteration is in $O\left(|\mathcal{S}|^2 |\mathcal{A}|\right)$ which can make the algorithm intractable for large problems. Recall that for pMDPs $|\mathcal{S}| \in O\left((|\Sigma|\,|\mathcal{R}|)^{\max d}\right)$ since in RANDAO's LRA case it means $|\mathcal{S}| \in O\left(\kappa^{2^{\kappa+1}}\right)$, any algorithm complexity containing $|\mathcal{S}|$ is not usable in our case. In the rest of the paper we also use the fact that $|\mathcal{A}| = \max d$ for pMDPs.

Policy extraction. (pseudocode in appendix 10, algorithm 4). In general, once the optimal value function V^* of an MDP $(\mathcal{S}, \mathcal{A}, P, R)$ is known, an optimal policy π^* can be computed with a complexity for a single state $s \in \mathcal{S}$ in $O\left(|\mathcal{S}|\,|\mathcal{A}|\right)$ (see appendix 10):

$$\pi^*(s) = a^* \in \arg\max_{a \in \mathcal{A}(s)} \left[R(s, a) + \gamma\, \mathbb{E}(V^*(s')|s, a)\right] \tag{2}$$

4.2 Reduced pMDPs

We use the previous notations for both reductions ex-post $(\mathcal{S}, \mathcal{A}, P, R)$ and ex-ante $(\Sigma, \mathrm{A}, P, \mathrm{R})$. In the ex-ante reduction, the sets A being sets of functions, their sizes are enormous and a naive value iteration would yield a terrible result. However, instead of iterating over the possible functions α (Bellman operator 1's max), we iterate over

becomes (proof in appendix 9.2):

$$T^*W(\sigma) = \mathop{\mathbb{E}}_{s\sim P(s|\sigma)} \left[\max_{a\in\mathbb{N}^*_{d(\sigma)}} R(s,a) + \gamma W(s^\sigma_a) \right] \tag{5}$$

The computation's complexity is in $O\left(|\Sigma|\,|\mathcal{S}|\,\max d\right)$ (see algorithm 5 in appendix 10) which reduces the iteration complexity by a factor of $\lambda = |\mathcal{S}|/|\Sigma|$ from the natural MDP (the ex-post MDP). In the card game case $\lambda > 40^8/4$, while for the LRA λ is of the order of $\kappa^{2^{\kappa+1}-1}$, this is gigantic, yet still entirely insufficient.

Policy Extraction. This result uses a point of view in between the ex-ante and ex-post reductions. Its advantage comes from the fact that reduced pMDPs have either a large state space $\mathcal{S}$ (ex-post reduction) or a large action space A (ex-ante reduction). The hybrid view allows us to better circumvent the scanning of those sets. It can be extended to Policy Extraction where given the ex-ante state values W, one can compute an optimal action in an ex-post state $s \in \mathcal{S}$:

$$\pi^*(s) = a^* \in \mathop{\arg\max}_{a\in\mathbb{N}^*_{d(\sigma)}} \left[s^R_a + \gamma W^*(s^\sigma_a) \right] \tag{6}$$

Once in s, the agent only considers actions in $\mathcal{A}(s)$, and only needs to know W which is of size Σ. Extracting an optimal action has a time complexity in $O\left(d(s)\right)$ and a space one in $O\left(|\Sigma|\right)$. In the card game example: $|\Sigma| = 4$ and $|\mathcal{A}(s)| \leq 8$, this is extremely economic in time and space, even a human could implement this[3]. This is also extremely useful for embedded systems with strong computation and memory constraints.

Equations 5, 6 give this theorem (proof is in appendix 9.2):

Theorem 4.1. *Given a pMDP* $(\Sigma, \mathcal{R}, P, d, c)$*, and its reduced MDPs* $(\mathcal{S}, \mathcal{A}, P, R)$ *and* $(\Sigma, \mathrm{A}, P, \mathrm{R})$*. There exist a dynamic programming implementation of value iteration of the ex-ante reduction whose complexity is in* $O\left(|\Sigma|\,|\mathcal{S}|\,\max d\right)$ *using the Bellman operator as in equation 5. One can then extract an optimal action in* $O\left(\max d\right)$*.*

Note that given the size of $\mathcal{S}$, see Table 2, this is not satisfactory yet for RANDAO's LRA.

[3]In fact, casino card counters use a similar trick, they only maintain a condensed information about the state of the dealer's deck (the ex-ante state), and adapt their action $a \in \mathcal{A}(s)$ accordingly within a large ex-post state set using simple decision rules

and for $s \in \mathcal{S}$*:*

$$\pi^*(s) = a^* \in \mathop{\arg\max}_{a\in\mathcal{A}(s)} \left[R(s,a) + \gamma\,\mathbb{E}(W^*(\sigma')|s,a) \right] \tag{8}$$

Computations are respectively in $O\left(|\mathcal{S}|\,|\Sigma|^2\,|\mathcal{A}|\right)$ *and* $O\left(|\Sigma|\,|\mathcal{A}|\right)$

The proof is in appendix 9.1, and is similar to that of Theorem 4.1. The difference in complexity comes from the fact that where we could directly access $W(\sigma')$ in reduced pMDP thanks to the deterministic transition from (s,a) to $\sigma' = s^\sigma_a$, we now have to compute $\mathbb{E}(W(\sigma')|s,a)$. The adapted pseudocode is in appendix 10, algorithms 3 and 6. The needed hypothesis represents an intermediate step during the Markov transition: when in state $s \in \mathcal{S}$ with $a \in \mathcal{A}(s)$ chosen, we first draw $\sigma' \in \Sigma$ with $P(\sigma'|s,a)$. This σ contains all the information needed to draw $s' \in \mathcal{S}$ (with probability $P(s'|\sigma')$). This structure is exactly that of repeated two-stage stochastic control problems where the σ-states represent possible first-stage situations and the s-states represents the second stage (often named *recourse*, see [23]).

4.4 Monte Carlo Value Iteration (MCVI)

Reduced pMDPs and MDPs from theorem 4.2 can use the values W of the ex-ante states as a heuristic which can be used, for instance, in an adapted Monte Carlo Value Iteration (MCVI)[4]. Here, we sample the ex-post states s from each possible ex-ante state σ. In each of the sampled ex-post state s, the value $V(s) = \max_{a\in\mathcal{A}(s)} R(s,a) + \gamma\,\mathbb{E}(W(\sigma')|s,a)$ is computed and then averaged to update $W(\sigma)$. The pseudocode can be found in appendix 10, algorithm 7 , it leads to an iteration complexity of $O\left(|\Sigma|^2\,|\mathcal{A}|\,n_{sample}\,\varsigma\right)$ where ς represents the sampling cost and allows one to bypass scanning $\mathcal{S}$ altogether. The cost of sampling for MDPs is usually in $O\left(1\right)$ but for pMDPs sampling s naturally from σ requires $d(\sigma) = d(s) = |\mathcal{A}(s)|$ individual sampling of (σ', r). While this works perfectly for the card game, where $\max d = |\mathcal{A}| = 8$, for the LRA we have an issue since $\max d = 2^\kappa$, all in all this gives for the LRA a complexity in $O\left(\kappa^2\,2^{2\kappa}\,n_{sample}\right)$. This is still an issue, especially if we want to investigate larger values for κ, additionally, with MCVI, we lose the convergence guarantees.

[4]MCVI comes in different flavors: for each sampled states, actions could be sampled with an exploratory policy. In our case we are exhaustive over actions. We focus on estimating expected values with sampling

to improve the computation of the Bellman operator T^*.

5.1 Changing the random variable:

In general, the Bellman equation expresses the static relation between the value of a state (the state value function V) and the expected value for the agent after a single stochastic transition (composed of the reward and the value function of the next state $r + \gamma V$). We already used the dual reductions to improve our computation of the Bellman operator (see equation 5), relying on the formulation of equation 3; we now want to leverage the IID draws to better compute the ex-post value function from as seen in equation 4.

Value functions represent a utility, and in equation 4, in ex-post state $s \in \mathcal{S}$, the agent following an optimal policy will pick the best draw s_a with $a \in \mathcal{A}(s)$. This means that the agent will maximize the utility $u_W(s,a) := s_a^r - c(a) + \gamma\, W(s_a^\sigma)$ associated with a certain unitary draw given his perception of the ex-ante state value function W. When W^*, this leads to an optimal action choice. We note $u_W^*(s) := \max_{a\in\mathcal{A}(s)} u_W(s,a)$ the ex-post maximal utility, this can be seen as a way to compute the ex-post state value function V of the ex-post state s for an a priory ex-ante state value function W. Combined with equation 3 this allows us to rewrite the ex-ante Bellman operator from equation 5 as:

$$T^*W(\sigma) = \mathop{\mathbb{E}}_{s\sim P(s|\sigma)} \left[u_W^*(s)\right], \text{ with } u_W^*(s) = \max_{a\in\mathbb{N}^*_{d(s)}} u_W(s,a) \quad (9)$$

Instead of seeing $s \sim P(s|\sigma)$ as the random variable from the ex-ante – ex-post transition, we will consider the realization of $u_W^*(s)$, written as u as being the new random variable. This means that the computation of the Bellman operator can be done by computing the expected utility u:

$$T^*W(\sigma) = \mathop{\mathbb{E}}_{u\sim P^W(u|\sigma)} [u] \quad (10)$$

The ex-post optimal utility u is *realized* by the stochastic transition between ex-ante state and ex-post state. It is a **max** of several $u_W(s,a)$ that are IID random variables once adjusted for the cost function's shift. This means that for a prior ex-ante state value function W and for a given ex-ante state σ, and actions $a_0, a_1 \in \mathcal{A}$, we have a link between the distributions of the realized utilities $u_W(s,a)$ of those actions: $P(u_W(s,a_0) + c(a_0)|\sigma) = P(u_W(s,a_1) + c(a_1)|\sigma)$. We note $\mathcal{U}$ the set of value u can take (its support). Given $P^W(u|\sigma)$ to be able to compute the Bellman operator as in equation 10. This is shown formally in appendix 9.3, and this is a simple computation in $\mathcal{O}(|\mathcal{U}|)$ which for our cases is in $\mathcal{O}(|\Sigma|\,|\mathcal{R}|)$ or in $\mathcal{O}(\kappa^2)$ as just discussed.

For a prior ex-ante state value function W, the distribution of u can be computed through brute force with its definition by enumerating all possible ex-post states $s \in \mathcal{S}$, that is: $P^W(u|\sigma) = \sum_{s\in\mathcal{S}} P(s,u|\sigma) = \sum_{s\in\mathcal{S}} P(s|\sigma)P(u|\sigma,s)$. Additionally we can write that: $P(u|\sigma,s) = \mathbb{1}\left[u_W^*(s) = u\right]$ because u is deterministic given s. Therefore, we can compute the distribution of the utility u with: $P^W(u|\sigma) = \sum_{s\in\mathcal{S}} P(s|\sigma)\mathbb{1}\left[u_W^*(s) = u\right]$. But this naive computation, which can be found in appendix 10, algorithm 8 would yield a complexity in $\mathcal{O}(|\Sigma|\,|\mathcal{S}|\max d)$. With it, we do not reach any improvement.

5.2 Efficient computation of the utility distribution:

This is where we finally leverage the relationship between W^* and d to improve the computation of $P^W(u|\sigma)$. We start by using cumulative distributions: $P^W(u \le v|\sigma)$ for $v \in \mathcal{U}$. This allows us to use the properties of the distribution of the max function[5] in the Bellman operator. Finally, we introduce the notation $P^W_{i,j}(u \le v|\sigma) = P^W(\max_{i\le a<j} u_W(s,a) \le v|\sigma)$ representing the probability that u is smaller than v when restraining the agent's choice to draws of index a with $1 \le i \le a < j \le d(\sigma)+1$. With this we can realize a dichotomy on the computation of $P^W(u \le v|\sigma)$ since, in general:

$$P^W_{1,i}(u \le v|\sigma) = P^W_{1,j}(u \le v|\sigma)P^W_{j,i}(u \le v|\sigma) \quad (11)$$

This dichotomy however does not provide automatically a faster computation. Let us look at the first case where $j = i+1$. The only thing that makes $P^W_{1,2}(u \le v|\sigma)$ different from $P^W_{i,i+1}(u \le v|\sigma)$ is the cost $c(i)$ versus the cost of $c(1)$ (as mentioned in the previous section). This means that we can deduce one from the other only with an indexing change in $\mathcal{U}$:

$$P^W_{i,i+1}(u \le v|\sigma) = P^W_{1,2}(u \le v + c(i) - c(1)|\sigma)$$

Unfortunately, without additional assumptions, this only holds for extracted draws of size 1, that is when $j = i+1$. However, with certain cost functions, this works for any extraction size. For a constant cost, this is trivial since the distribution is the same. Let us

[5] Given two numerical random variables X and Y, and a number z, we have: $P(\max(X,Y) \le z) = P(X \le z)P(Y \le z)$

$$P^W_{i,j}(u \le v|\sigma) = P^W_{1,j-i+1}(u \le v + c_1(i-1)|\sigma)$$

In general, we need a function $\varsigma : i \in \mathbb{N} \mapsto \varsigma(i) \in \mathbb{R}$ such that the distributions over the sub-intervals can be deduced from each other. For instance when splitting $\mathbb{N}^*_{2i}$ in $\mathbb{N}^*_i$ and $\mathbb{N}^*_i + i$, we need the function to allow us to shift the extraction interval $[i+1,\ 2i]$ in the following way:

$$P^W_{i+1,2i+1}(u \le v|\sigma) = P^W_{1,i+1}(u \le v + \varsigma(i)|\sigma) \tag{12}$$

Which gives us:

$$P^W_{1,2i+1}(u \le v|\sigma) = P^W_{1,i+1}(u \le v|\sigma)\, P^W_{1,i+1}(u \le v + \varsigma(i)|\sigma) \tag{13}$$

Note that since the numbering of the IID draws is arbitrary, this can be extended to the existence of a function ς for permutation of $\mathbb{N}^*_{d(\sigma)}$.

For the LRA on RANDAO, this works with $\varsigma = 1$ when splitting $\mathbb{N}^*_{2i}$ in the middle as previously. This comes from the fact that controlling an additional block at the end of an epoch doubles the number of possible seeds to choose from. The new ones result from having not proposed this additional block, and then having the same proposition configurations as before for the remaining blocks. That is why $d(\sigma)$ is a power of two and why $\varsigma = 1$: the new possibilities have the same costs as before, plus 1 for the new discarded block.

Theorem 5.1. *Using the given notations, if a pMDP satisfies the condition from equation 12, then we can compute the Bellman operator for the ex-ante states in $O\left(|\Sigma|^2\, |\mathcal{R}|\, \max d\, \log(\max d)\right)$. Additionally, if $\mathcal{U}$ is smaller than the worst case ($|\mathcal{U}| \in O\left(|\Sigma|\, |\mathcal{R}|\, \max d\right)$), we have a complexity in $O\left(|\mathcal{U}|\, |\Sigma|\, \log(\max d)\right)$.*

The pseudocode of the computation can be found in appendix 10, algorithm 9. **We end up with a significant improvement in the case of the LRA: the complexity has been lowered from the initial $O\left(2^\kappa \kappa^{2^{\kappa+2}}\right)$ to a polynomial complexity $O\left(\kappa^4\right)$.** That is because we have $|\Sigma| = \kappa + 1$, but also $\max d = 2^\kappa$, and $|\mathcal{U}| \in O\left(\kappa^2\right)$

6 VANISHING DISCOUNT FACTOR

The previous ways to solve MDPs rely on the use of the discount factor γ which adjusts the value of future rewards compared to that of the present rewards. In the case of the card game or the LRA, the discount factor can help represent the fact that the repetition of the game has an unknown end (a patch preventing the LRA for example). However, if one supposes that the repetition is infinite, $O\left((1-\gamma)^{-1}\right)$. We refer the reader to [20], theorems 9.28 and 9.29, to see in details how this can be handled. In short, we do not look at the state values, but at the relative state values. That is, given a reference ex-ante state σ_0, we look at the relative value function $\Delta W^*_{\sigma_0}(\sigma) := W^*(\sigma) - W^*(\sigma_0)$ which converges when $\gamma \to 1$ for a large class of MDPs in which ergodic[6] MDPs are part of, and our problems are ergodic. In practice, this allows to reach convergence in very few iterations (6 for the cards, 10 for LRA). The adapted Bellman operator is:

$$\begin{aligned} T^*\, \Delta W_{\sigma_0}(\sigma) = & \mathop{\mathbb{E}}_{s \sim P(s|\sigma)} \left[\max_{a \in \mathbb{N}^*_{d(\sigma)}} R(s,a) + \gamma \Delta W_{\sigma_0}(s^\sigma_a) \right] \\ & - \mathop{\mathbb{E}}_{s \sim P(s|\sigma_0)} \left[\max_{a \in \mathbb{N}^*_{d(\sigma_0)}} R(s,a) + \gamma \Delta W_{\sigma_0}(s^\sigma_a) \right] \end{aligned} \tag{14}$$

And the adapted policy extraction:

$$\pi^*(s) = a^* \in \mathop{\arg\max}_{a \in \mathbb{N}^*_{d(\sigma)}} \left[s^R_a + \gamma \Delta W^*_{\sigma_0}(s^\sigma_a) \right] \tag{15}$$

7 RESULTS

The code for computing the following results is available at [4].

7.1 Optimal ex-ante state value function W^*

In Table 3, one can see the relative ex-ante state values $\Delta W^*_{\sigma'}(\sigma)$ for $\gamma = 1$ of our two problems (we stopped at $\sigma = 3$ for the LRA, but in [4], one can see the results for higher σ). Those tables enable the agent to compute the optimal strategy with equation 15. For instance, we see that the club-diamond relative value is 2.01, this means that picking a 7 of diamond is worth slightly more than a 9 of club (without considering costs). Similarly, if revealing all blocks would yield $\sigma' = 1$ (only 1 block controlled at the end of the next epoch) an attacker would be willing to lower his net reward by 1 only if it allows him to get at least the last 3 blocks instead (2 would be insufficient as the relative value $\sigma = 1$ to $\sigma = 2$ is 0.97, but the 1-3 relative value is 1.79).

[6] an MDP where all states are recurrent, no matter the policy

7.3 Attack Performance

The obtained results in terms of attack performance are unimpressive but in line with the previous results of [3], which confirms the validity of our methodology. As they did, we compare the optimal attack to:

- The myopic strategy: the attacker simply maximize the instant reward, it is myopic in the sense that it is reached for $\gamma = 0$, i.e. no value is given to future rewards.
- The control-max strategy: the attacker maximize the number of blocks controlled at the end of the next epoch and ignores rewards.
- The honest strategy: no blocks are withheld.

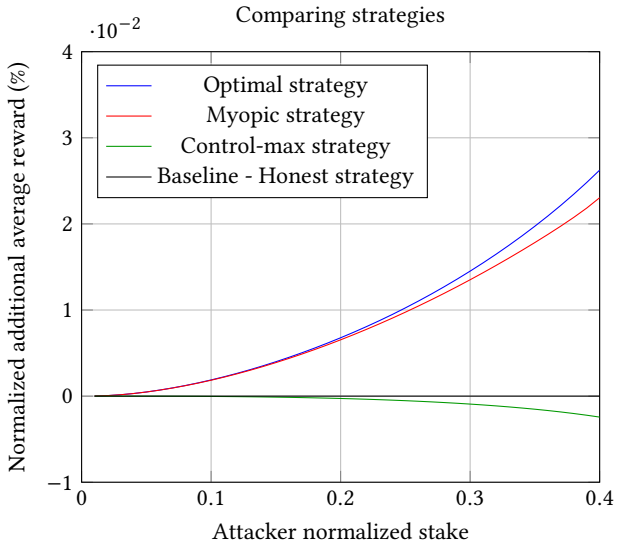

Figure 2: Normalized additional average reward (percentage) as a function of relative stake for different strategies, $\kappa = 32$

Figure 2 shows that the optimal strategy offers only marginal gains over the myopic policy, and both deliver weak profitability. Hence a rational attacker would likely prefer the simpler, less bug-prone myopic approach; the optimal attack also bears higher financial risk by sometimes accepting lower current rewards. Considering additional deterrents, e.g., social slashing (community-enforced reputational or economic penalties), these attacks appear

with a reversed decision flow such as the Last Reveaer Attack (LRA) in Ethereum's RANDAO which was our case study. We reduced pMDPs to standard MDPs in two ways, then by leveraging the dual perspective of those reductions, we derived theoretical results enabling optimal policy computation and deeper insight into attacker strategies. We also identified conditions under which these methods apply to broader MDP subclasses, including settings where the discount factor plays a limited role—enhancing the applicability of pMDPs. This allowed us to reduce value iteration complexity per iteration from the initial $O\left(2^{\kappa}\kappa^{2^{\kappa+2}}\right)$ to a polynomial complexity $O\left(\kappa^4\right)$, and to have it converge in just a few iterations.

Empirically, our results align with this picture on three dimensions. *Value computation.* We computed optimal ex-ante state values W^* (Section 7.1), which directly yield optimal policies via Equation 15 and quantify trade-offs between immediate net reward and end-of-epoch control. *Runtime.* The variable-change value iteration shows iteration times consistent with the $O\left(\kappa^4\right)$ claim (Table 4) once overheads are taken into account. *Strategy comparisons.* As reported in Figure 2, the optimal policy offers only marginal gains over the myopic policy ($\gamma = 0$), and both yield weak profitability relative to the honest baseline. A rational attacker would therefore likely adopt the simpler myopic strategy, avoiding the additional implementation and financial risk of an "optimal" attack that occasionally sacrifices current rewards. Considering extra-protocol penalties (social slashing), such attacks are further disincentivized. We also evaluated a control-max policy as a proxy for multi-attacker dynamics; its cost is modest and it can act as a viable defensive heuristic (e.g., keeping last-block control to deny seed manipulation to others).

Overall, pMDPs provide a practical and interpretable tool for large, combinatorial decision problems where future control matters. Beyond the LRA, the same methodology applies to MDP subclasses with similar information structure such as the taxi ride selection problem presented in introduction. Limitations include sensitivity to cost function specifications, the limited size of the reversed flow decision problem class. Promising directions include: scaling via function approximation or learning; incorporating heterogeneous agents and explicit multi-agent interactions across epochs as well as stress-testing defensive policies (e.g., control-max variants) in adversarial simulations. We hope the combination of theory, runtime evidence, and strategy comparisons provides a useful basis for security assessment and mechanism design in proof-of-stake systems and related settings.

cbeci [Online; accessed 11. Dec. 2024].
[3] Kaya Alpturer and S. Matthew Weinberg. 2024. Optimal RANDAO Manipulation in Ethereum. arXiv:2409.19883 [cs.GT] https://arxiv.org/abs/2409.19883
[4] Anonymous. 2024. Anonymized Repository - Anonymous GitHub. https://anonymous.4open.science/r/pMDPs-8B1B/README.md [Online; accessed 15. May 2025].
[5] Richard Bellman. 1957. A Markovian Decision Process. *Indiana Univ. Math. J.* 6 (1957), 679–684. Issue 4.
[6] R.E. Bellman and Rand Corporation. 1957. *Dynamic Programming*. Princeton University Press. https://books.google.fr/books?id=rZW4ugAACAAJ
[7] Dan Boneh, Joseph Bonneau, Benedikt Bünz, and Ben Fisch. 2018. Verifiable Delay Functions. Cryptology ePrint Archive, Paper 2018/601. https://eprint.iacr.org/2018/601 https://eprint.iacr.org/2018/601.
[8] Craig Boutilier, Richard Dearden, and Moisés Goldszmidt. 2000. Stochastic dynamic programming with factored representations. *Artificial Intelligence* 121, 1 (2000), 49–107. https://doi.org/10.1016/S0004-3702(00)00033-3
[9] Giorgia Casella, Barbara Bigliardi, Serena Filippelli, and Eleonora Bottani. 2023. Cases of application of blockchain on the supply chain: a literature review. *Procedia Computer Science* 217 (2023), 1416–1426. https://doi.org/10.1016/j.procs.2022.12.340 4th International Conference on Industry 4.0 and Smart Manufacturing.
[10] Peter Fairley. 2019. Ethereum Plans to Cut Its Absurd Energy Consumption by 99 Percent. https://trum.ieee.org/computing/networks/ethereum-plans-to-cut-its-absurd-energy-consumption-by-99-percent
[11] Ethereum Foundation. [n. d.]. Ethereum 2.0 Specifications. https://github.com/ethereum/eth2.0-specs
[15] Jiajun Liu and Junhao Wu. 2024. A Comprehensive Survey on Blockchain Technology and Its Applications. *Highlights in Science, Engineering and Technology* 85 (03 2024), 128–138. https://doi.org/10.54097/r0ggyr24
[16] Satoshi Nakamoto. 2009. Bitcoin: A Peer-to-Peer Electronic Cash System. (May 2009). http://www.bitcoin.org/bitcoin.pdf
[17] Team Near. 2024. Consensus | NEAR Protocol Specification. https://nomicon.io/ChainSpec/Consensus [Online; accessed 11. Dec. 2024].
[18] Martin L Puterman. 2014. *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons.
[19] Team Randao. 2017. Randao: Verifiable random number generation. White Paper.
[20] Eilon Solan. 2022. *A Course in Stochastic Game Theory*. Cambridge University Press.
[21] Do Hai Son, Tran Thi Thuy Quynh, and Le Quang Minh. 2024. RANDAO-based RNG: Last Revealer Attacks in Ethereum 2.0 Randomness and a Potential Solution. arXiv:2403.09541 [cs.CR] https://arxiv.org/abs/2403.09541
[22] Richard S. Sutton and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction* (second ed.). The MIT Press. http://incompleteideas.net/book/the-book-2nd.html
[23] Ken Tomiyama. 1985. Two-stage optimal control problems and optimality conditions. *Journal of Economic Dynamics and Control* 9, 3 (1985), 317–337. https://doi.org/10.1016/0165-1889(85)90010-7
[24] Toni Wahrstätter. 2023. Selfish Mixing and RANDAO Manipulation - Consensus - Ethereum Research. ethresear.ch/t/selfish-mixing-and-randao-manipulation/16081. (Accessed on 10/13/2023).

using the conditional independence and the Markov property, we have for $\sigma' \in \Sigma$:

$$\begin{aligned}
W^*(\sigma') &= \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_t | \sigma_0 = \sigma', \pi^*\right] \\
&= \sum_{s' \in \mathcal{S}} P(s'|\sigma', \pi^*)\, \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_t | s_0 = s', \sigma_0 = \sigma', \pi^*\right] \\
&= \sum_{s' \in \mathcal{S}} P(s'|\sigma')\, \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_t | s_0 = s', \pi^*\right] \\
&= \mathbb{E}\left[V^*(s')|\sigma'\right]
\end{aligned}$$

$$\pi^* = s \in \mathcal{S} \mapsto a^* \in \underset{a \in \mathcal{A}(s)}{\arg\max} \left[s_a^R + \gamma\, \mathbb{E}\left(V^*(s')|s, a\right)\right]$$

can indeed be re-written:

$$\pi^* = s \in \mathcal{S} \mapsto a^* \in \underset{a \in \mathcal{A}(s)}{\arg\max} \left[s_a^R + \gamma\, \mathbb{E}\left(W^*(\sigma')|s, a\right)\right]$$

□

$$
\begin{aligned}
&= \max_{\alpha \in \mathrm{A}(\sigma)} \left[\sum_{s \in \mathcal{S}} \left[P(s|\sigma) R(s, \alpha(s)) \right] + \gamma \sum_{\sigma' \in \Sigma} \sum_{s \in \mathcal{S}} \left[P(\sigma', s|\sigma, \alpha)\, W(\sigma') \right] \right] \\
&= \max_{\alpha \in \mathrm{A}(\sigma)} \left[\sum_{s \in \mathcal{S}} \left[P(s|\sigma) R(s, \alpha(s)) \right] + \gamma \sum_{\sigma' \in \Sigma} \sum_{s \in \mathcal{S}} \left[P(s|\sigma) P(\sigma'|s, \alpha(s))\, W(\sigma') \right] \right] \\
&= \max_{\alpha \in \mathrm{A}(\sigma)} \left[\sum_{s \in \mathcal{S}} P(s|\sigma) \left[R(s, \alpha(s)) + \gamma \sum_{\sigma' \in \Sigma} P(\sigma'|s, \alpha(s))\, W(\sigma') \right] \right] \\
&= \sum_{s \in \mathcal{S}} P(s|\sigma) \max_{\alpha \in \mathrm{A}(\sigma)} \left[R(s, \alpha(s)) + \gamma \sum_{\sigma' \in \Sigma} P(\sigma'|s, \alpha(s))\, W(\sigma') \right] \\
&= \sum_{s \in \mathcal{S}} P(s|\sigma) \max_{a \in \mathcal{A}(s)} \left[R(s, a) + \gamma \sum_{\sigma' \in \Sigma} P(\sigma'|s, a)\, W(\sigma') \right] \\
&= \mathop{\mathbb{E}}_{s \sim P(s|\sigma)} \left[\max_{a \in \mathcal{A}(s)} \left[R(s, a) + \gamma \sum_{\sigma' \in \Sigma} P(\sigma'|s, a)\, W(\sigma') \right] \right] \\
&= \mathop{\mathbb{E}}_{s \sim P(s|\sigma)} \left[\max_{a \in \mathcal{A}(s)} \left[R(s, a) + \gamma \sum_{\sigma' \in \Sigma} \mathbb{1} \left[\sigma' = s_a^\sigma \right]\, W(\sigma') \right] \right] \\
&= \mathop{\mathbb{E}}_{s \sim P(s|\sigma)} \left[\max_{a \in \mathcal{A}(s)} \left[R(s, a) + \gamma W(s_a^\sigma) \right] \right]
\end{aligned}
$$

$$
\begin{aligned}
\underset{u\sim P^W(u|\sigma)}{\mathbb{E}}(u|s) &= \sum_{u\in\mathcal{U}} P(u|s)\, u && (16)\\
&= \sum_{u\in\mathcal{U}} \sum_{s\in\mathcal{S}} P(s|\sigma)\mathbb{1}\left[u^*_W(s)=u\right] u && (17)\\
&= \sum_{s\in\mathcal{S}} P(s|\sigma) \sum_{u\in\mathcal{U}} \mathbb{1}\left[u^*_W(s)=u\right] u && (18)\\
&= \sum_{s\in\mathcal{S}} P(s|\sigma)\, u^*_W(s) && (19)\\
&= \sum_{s\in\mathcal{S}} P(s|\sigma) \max_{a\in\mathcal{A}(s)} \left[R(s,a)+\gamma\,\mathbb{E}(W(\sigma')|s,a)\right] && (20)\\
&= \sum_{s\in\mathcal{S}} P(s|\sigma) \max_{\alpha\in\mathrm{A}(\sigma)} \left[R(s,\alpha(s))+\gamma\,\mathbb{E}(W(\sigma')|s,\alpha(s))\right] && (21)\\
&= \max_{\alpha\in\mathrm{A}(\sigma)} \sum_{s\in\mathcal{S}} P(s|\sigma) \left[R(s,\alpha(s))+\gamma\,\mathbb{E}(W(\sigma')|s,\alpha(s))\right] && (22)\\
&= \max_{\alpha\in\mathrm{A}(\sigma)} \left[\mathbb{E}(R(s,\alpha(s))|\sigma)+\gamma\,\mathbb{E}(W(\sigma')|\sigma,\alpha)\right] && (23)\\
&= \max_{\alpha\in\mathrm{A}(\sigma)} \left[R(\sigma,\alpha)+\gamma\,\mathbb{E}(W(\sigma')|\sigma,\alpha)\right] && (24)\\
&= T^*W(\sigma) && (25)
\end{aligned}
$$

□

10 ALGORITHMS

In this appendix, one can find the pseudocode for all algorithms mentioned in the paper. We repeatedly use the fact that for a pMDP $(\Sigma, \mathcal{R}, P, d, c)$, with a reduced ex-post state set $\mathcal{S}$ and reduced action set $\mathcal{A}$ we have $\mathcal{S} \in \mathcal{O}\left((|\Sigma|\,|\mathcal{R}|)^{\max d}\right)$ and $|\mathcal{A}| = \max d$.

```
    V'[s] := max(V'[s], v_{s,a})
  end for
end for
until ||V − V'|| ≤ ε
Output: Value vector V
```

Algorithm 1: Loops on $\mathcal{S}$, $\mathcal{A}(s)$, $\mathcal{S}$. Complexity in $\mathcal{O}\left(|\mathcal{S}|^2\,|\mathcal{A}|\right)$

Algorithm 2 Value iteration for reduced pMDPs

```
Input: pMDP (Σ, R, P, d, c), ex-post state set S, distributions
P(s|σ), discount factor γ, convergence precision ε
Initialize W := 0, W' := 0  Vectorized versions of the ex-ante value
function
repeat
  W := W'
  W' := 0
  for σ ∈ Σ                 computing W' = T*W, using equation 5
    for s ∈ S                                  computing one element
    W'[σ] = E_{s~P(s|σ)} [max_{a∈A(s)} R(s,a) + γW(s_a^σ)]
      V := −∞
      for a ∈ N*_{d(σ)}                 computing a possible outcome
      max_{a∈A(s)} R(s,a) + γW(s_a^σ)
        V := max(V[s], s_a^r − c(a) + γW[s_a^σ])        Note that for
        reduced pMDPs R(s,a) = s_a^r − c(a)
      end for
      W'[σ] += P(s|σ) V
    end for
  end for
until ||W − W'|| ≤ ε
Output: Value vector W
```

Algorithm 2: Loops on $\Sigma, \mathcal{S}, \mathcal{A} = \mathbb{N}^*_{d(\sigma)}$. Complexity in $\mathcal{O}\left(|\Sigma|\,|\mathcal{S}|\,|\mathcal{A}|\right) \subset \mathcal{O}\left(|\Sigma|(|\Sigma|\,|\mathcal{R}|)^{\max d}\ \max d\right)$

$V' := \max_{a \in \mathcal{A}(s)} R(s,a) + \gamma\, \mathbb{E}_{\sigma' \sim P(\sigma'|s,a)} [W(\sigma')]$

$V' := R(s,a)$ *computing*

$V' := R(s,a) + \gamma\, \mathbb{E}_{\sigma' \sim P(\sigma'|s,a)} [W(\sigma')]$

for $\sigma' \in \Sigma$ *adding the term in* $\gamma\, \mathbb{E}_{\sigma' \sim P(\sigma'|s,a)} [W(\sigma')]$

$V' \mathrel{+}= \gamma\, P(\sigma'|s,a)\, \mathbf{W}[\sigma']$

end for

$V := \max(V, V')$

end for

$\mathbf{W}'[\sigma] \mathrel{+}= P(s|\sigma)\, V$

end for

end for

until $||\mathbf{W} - \mathbf{W}'|| \leq \varepsilon$

Output: Value vector $\mathbf{W}$

Algorithm 3: Loops on $\Sigma, \mathcal{S}, \mathcal{A}, \Sigma$. Complexity in $O\left(|\Sigma|^2\, |\mathcal{S}|\, |\mathcal{A}|\right)$

Algorithm 4 Standard Policy extraction for a single state

Input: MDP $(\mathcal{S}, \mathcal{A}, P, R)$, discount factor γ, Value vector $\mathbf{V}$, state $s \in \mathcal{S}$

Initialize $V' := -\infty$

for $a \in \mathcal{A}(s)$ *computing* $a^* \in \arg\max_{a \in \mathcal{A}(s)} R(s,a) + \gamma\, \mathbb{E}_{s' \sim P(s'|s,a)} [V(s')]$

Initialize $v_{s,a} := R(s,a)$

for $s' \in \mathcal{S}$ *adding the term* $\gamma\, \mathbb{E}_{s' \sim P(s'|s,a)} [V(s')]$

$v_{s,a} \mathrel{+}= \gamma P(s'|s,a)\, \mathbf{V}[s']$

end for

if $v_{s,a} > V'$ **then**

$a^* := a$

$V' := v_{s,a}$

end if

end for

Output: action a^*

end if

end for

Output: action a_{best}

Algorithm 5: Loops on $\mathcal{A}$. Complexity in $O\left(|\mathcal{A}|\right) \subset O\left(\max d\right)$

Algorithm 6 Policy extraction for MDPs of theorem 4.2 for a single state

Input: MDP $(\mathcal{S}, \mathcal{A}, P, R)$, set Σ, distributions $P(\sigma'|s,a)$, discount factor γ, Value vector $\mathbf{W}$, state $s \in \mathcal{S}$

Initialize $W' := -\infty$

for $a \in \mathcal{A}(s)$ *computing* $a^* \in \arg\max_{a \in \mathcal{A}(s)} R(s,a) + \gamma\, \mathbb{E}_{\sigma' \sim P(\sigma'|s,a)} [W(\sigma')]$

Initialize $w_{s,a} := R(s,a)$

for $\sigma' \in \Sigma$ *adding the term* $\gamma\, \mathbb{E}_{\sigma' \sim P(\sigma'|s,a)} [W(\sigma')]$

$w_{s,a} \mathrel{+}= \gamma P(\sigma'|s,a)\, \mathbf{W}[\sigma']$

end for

if $w_{s,a} > W'$ **then**

$a_{best} := a$

$W' := w_{s,a}$

end if

end for

Output: action a_{best}

Algorithm 6: Loops on $\mathcal{A}, \Sigma$. Complexity in $O\left(|\Sigma|\, |\mathcal{A}|\right)$

sampling $s = (s_i)_{i \in \mathbb{N}^*_{d(\sigma)}}$ is in $\mathcal{O}(d(\sigma))$
for $a \in \mathcal{A}(s)$ *computing* $\max_{a \in \mathcal{A}(s)} R(s,a) + \gamma\, \mathbb{E}_{\sigma' \sim P(\sigma'|s,a)}[W(\sigma')]$
Initialize $w_{s,a} := R(s,a)$
for $\sigma' \in \Sigma$ *adding the term* $\gamma\, \mathbb{E}_{\sigma' \sim P(\sigma'|s,a)}[W(\sigma')]$
$w_{s,a} \mathrel{+}= \gamma P(\sigma'|s,a)\, \mathbf{W}[\sigma']$ *For reduced pMDPs, skip the loop with* $w_{s,a} = s^r_a - c(a) + \mathbf{W}[s^\sigma_a]$
end for
$\mathbf{W}'[\sigma] \mathrel{+}= w_{s,a}$
end for
end for
$\mathbf{W}'[\sigma] := \mathbf{W}'[\sigma]/n_{sample}$
end for
$\mathbf{W} := \mathbf{W}'$
until $||W - W'|| \leq \varepsilon$
Output: Value vector $\mathbf{W}$

Algorithm 7: Loops on $\Sigma, \mathbb{N}^*_{n_{sample}}, \mathcal{A}, \Sigma$. Therefore the complexity is in $\mathcal{O}\left(|\Sigma|^2\, |\mathcal{A}|\, n_{sample}\, \varsigma\right)$ (where ς is the sampling cost). For reduced pMDP the innermost loop can be skipped since we can directly compute $w_{s,a} = s^r_a - c(i) + \gamma\, \mathbf{W}[s^\sigma_a]$. This allows us to lower the complexity to $\mathcal{O}\left(|\Sigma|\, |\mathcal{A}|\, n_{sample}\, \varsigma\right) \subset \mathcal{O}\left(|\Sigma|\, \max d\, n_{sample}\, \varsigma\right)$.

Algorithm 8 Naïve computation of $P^W(u|\sigma)$ for reduced pMDPs

Input: pMDP $(\Sigma, \mathcal{R}, P, d, c)$, Value vector $\mathbf{W}$, discount factor γ
Initialize $\mathcal{U} = \{\}$
for $\sigma \in \Sigma,\ r \in \mathcal{R},\ i \in \mathbb{N}^*_{d(\sigma)}$ *We first compute* $\mathcal{U}$
$\mathcal{U}$.add$(r - c(i) + \gamma\, W(\sigma))$
end for
Initialize $\mathbf{P}^W := zeroes(|\Sigma|, |\mathcal{U}|)$ *Matrix of zeroes of size* $|\Sigma| * |\mathcal{U}|$
for $\sigma \in \Sigma$ *proceeding ex-ante state by ex-ante state*
for $s \in \mathcal{S}$ *Computing* $P^W[u|\sigma] = \sum_{s \in \mathcal{S}} P(s|a) \mathbb{1}\left[\max_{a \in \mathbb{N}^*_{d(\sigma)}} s^r_a - c(a) + \gamma\, \mathbf{W}[s^\sigma_a] = u\right]$
Initialize $u := -\infty$
for $a \in \mathbb{N}^*_{d(\sigma)} = \mathcal{A}(s)$ *Computing* $\max_{a \in \mathbb{N}^*_{d(\sigma)}} s^r_a - c(a) + \gamma\, \mathbf{W}[s^\sigma_a]$
$u := \max(u, s^r_a - c(i) + \gamma \mathbf{W}[s^\sigma_a])$
end for
$\mathbf{P}^W[\sigma, u] \mathrel{+}= P(s|\sigma)$
end for
end for
Output: Utility distribution $\mathbf{P}^W$

Input: pMDP $(\Sigma, \mathcal{R}, P, d, c)$, Value vector $\mathbf{W}$, discount factor γ, Probability adaptation functions ς
Initialize $\mathcal{U} = \{\}$
for $\sigma \in \Sigma,\ r \in \mathcal{R},\ i \in \mathbb{N}^*_{d(\sigma)}$ *We first compute* $\mathcal{U}$
$\mathcal{U}$.add$(r - c(i) + \gamma\, W(\sigma))$
end for
Initialize $\mathbf{P}^W = zeroes(|\Sigma|, |\mathcal{U}|)$ *Then we compute the probabilities*
for $\sigma \in \Sigma$
for $\sigma' \in \Sigma$ *We initialize* $\mathbf{P}^W[\sigma, \ .\ ]$ *as if* $d(\sigma) = 1$
for $r \in \mathcal{R}$
$u = r - c(1) + \gamma\, W(\sigma')$
$\mathbf{P}^W[\sigma, u] \mathrel{+}= P(\sigma', r|\sigma)$
end for
end for
The following loop computes $\mathbf{P}^W[\sigma, \ .\ ]$ *as if* $d(\sigma) = 2^i$
for $i \in \mathbb{N}^*_{\log_2(d(\sigma))}$ *For simplicity we consider only the cases where* $d(\sigma)$ *is a power of 2*
$\mathbf{P}^W[\sigma, :] := \mathbf{P}^W[\sigma, :] * \mathbf{P}^W[\sigma, : (+\varsigma(i))]$ *last indexation is the implementation of equation 12*
end for
end for
Output: Utility cumulative distributions $\mathbf{P}^W$

Algorithm 9: First we have the initial computation of $\mathcal{U}$, as explained for the previous algorithm, that is in $\mathcal{O}(|\mathcal{U}|) \subset \mathcal{O}(|\Sigma|\, |\mathcal{R}|\, \max d)$. Then we have loops on Σ, Σ, $\mathcal{R}$, this part is difficultly the bottleneck (for instance in the LRA's case it is in $\mathcal{O}(\kappa^3)$), so we will ignore it. Then there are loops on Σ, $\mathbb{N}^*_{\log_2(d(\sigma))}$. This gives a complexity in $\mathcal{O}(|\Sigma|\, \log |\mathcal{A}|\, |\mathcal{U}|) \subset \mathcal{O}(|\Sigma|\, \log(\max d)\, |\mathcal{U}|) \subset \mathcal{O}\left(|\Sigma|^2\, |\mathcal{R}|\, \max d\, \log(\max d)\right)$